



Olist E-Commerce ML Project

Model Comparison Report

Intelligent Business Analytics & Forecasting System — ML Mini Project

Covering: Predictive Modeling (Regression & Classification) · Customer Segmentation (Clustering) · NLP Review
Analysis · Time-Series Forecasting · End-to-End Pipelines with Cross-Validation

GitHub Repository: github.com/rahulkantode26/Olist-E-Commerce-Data-Analysis-Project

Introduction

This report is a consolidated model-comparison companion to the Olist E-Commerce Data Analysis notebook. The underlying project works with the Brazilian E-Commerce Public Dataset by Olist — real, anonymized orders placed between 2016 and 2018, spread across 9 linked CSV files (orders, items, products, customers, sellers, payments, reviews, geolocation, and a category-name translation table).

The notebook is organized around four ML pillars, each producing its own set of models that are trained, evaluated, and compared side by side:

- Predictive Modeling — Regression (delivery time) and Classification (late-delivery risk)
- Customer Segmentation — unsupervised clustering (KMeans vs. DBSCAN)
- NLP Review Analysis — sentiment classification from review text (TF-IDF + Logistic Regression, with Word2Vec and NER explored)
- Time-Series Sales Forecasting — ARIMA vs. SARIMA vs. Prophet

A final module rebuilds the top regression and classification models as proper scikit-learn Pipelines and re-validates them with cross-validation, before locking in the two production-ready final models. Every table below reproduces the actual metrics produced when the notebook was run, with the best score in each column highlighted in green.

1. Predictive Modeling — Regression

Goal: Predict `delivery_time_days`

Four regression models were trained to predict how many days an order will take to actually arrive, using order, item, product, payment, and location-derived features.

- Linear Regression — simplest baseline, fits a straight-line relationship between features and delivery time
- Ridge Regression — Linear Regression plus an L2 penalty that shrinks large coefficients to reduce overfitting
- Lasso Regression — like Ridge, but its L1 penalty can shrink some coefficients all the way to zero, acting like automatic feature selection
- Random Forest Regressor — an ensemble of decision trees that can capture non-linear patterns and feature interactions

Models are judged on:

- MAE (Mean Absolute Error) — average error in days, easy to interpret directly
- RMSE (Root Mean Squared Error) — like MAE but penalizes large mistakes more heavily
- R^2 (R-squared) — how much of the variation in delivery time the model explains (1.0 = perfect, 0 = no better than guessing the average)

Regression Model Comparison Table

Model	MAE (days)	RMSE (days)	R^2
Random Forest Regressor	5.000	7.933	0.275
Ridge Regression	5.293	8.170	0.231
Linear Regression	5.293	8.171	0.231
Lasso Regression	5.414	8.335	0.199

Best Model Justification

The Random Forest Regressor comes out ahead on all three metrics (lowest MAE, lowest RMSE, highest R^2). This makes sense: delivery time likely depends on non-linear interactions — for example "far-away state" + "heavy item" + "December" compounding together — which tree-based models capture naturally but straight-line models (Linear/Ridge/Lasso) cannot.

Note: the R^2 here is modest across all four models. Delivery time still carries a lot of real-world randomness — traffic, carrier delays, warehouse backlogs — that simply isn't present in the dataset, so no model here was ever going to be near-perfect. That's an expected and honest result, not a modeling failure.

Top Features Driving Delivery Time

Using the trained Random Forest Regressor's `feature_importances_`, the top 10 features were visualized to explain, in plain terms, what most strongly drives predicted delivery time — useful for communicating the model's logic to a non-technical audience (e.g. "delivery time is mostly driven by distance-related and product-weight features").

2. Predictive Modeling — Classification

Goal: Predict `is_late` (will this order arrive late?)

Four classification models were trained to flag orders at risk of arriving late. Only about 8% of orders in the dataset are actually late, so this is an imbalanced classification problem — `class_weight="balanced"` was applied wherever supported, so models pay extra attention to the rare "late" class instead of ignoring it in favor of the easy majority class.

- Logistic Regression — simple, interpretable linear baseline for classification
- Decision Tree — splits data into yes/no rules, easy to visualize and interpret
- Random Forest — an ensemble of many decision trees, usually more accurate and stable
- KNN (K-Nearest Neighbors) — classifies an order based on the "closest" similar historical orders

Models are judged on 5 metrics: Accuracy, Precision, Recall, F1 Score, and ROC-AUC (defined as: of orders predicted late, how many really were; of orders that were really late, how many were caught; the balance between the two; and how well the model ranks late orders as riskier across all thresholds, respectively).

Classification Model Comparison Table

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	0.907	0.352	0.202	0.257	0.756
Logistic Regression	0.631	0.131	0.646	0.217	0.688
KNN	0.914	0.344	0.085	0.136	0.651
Decision Tree	0.873	0.200	0.200	0.200	0.565

KNN posts the highest raw accuracy (0.914) and a competitive precision, but achieves this mostly by predicting the easy majority class — its recall is the worst of the four models (0.085), meaning it catches almost none of the genuinely late orders. High accuracy on an imbalanced target like this one is not, by itself, evidence of a good model.

Best Model Justification — A Nuanced Call

This one is worth stating carefully instead of just picking the top row of the table:

- Random Forest gets the best Accuracy and ROC-AUC, but even with `class_weight="balanced"`, it still catches relatively few actual late orders (low recall = 0.202). It's cautious about flagging "late," which keeps overall accuracy high but misses real problem orders.
- Logistic Regression has much lower accuracy, but noticeably higher recall (0.646) — it flags more orders as at-risk-of-being-late, catching more genuine late deliveries at the cost of more false alarms.

Which one is "best" genuinely depends on the business goal: if the cost of a missed late delivery is high (e.g. proactively warning customers or expediting shipping), Logistic Regression's higher recall is more valuable. If the goal is a high-confidence, low-noise ranking of risk (e.g. feeding a downstream system that can't tolerate many false alarms), Random Forest's higher ROC-AUC and precision are preferable.

Confusion Matrices & Feature Importance

Confusion matrices for all four classifiers were plotted side by side to show exactly where each model's predictions went right or wrong (correctly predicted on-time vs. late, and the two types of mistakes). Random Forest's feature importances were also visualized to show the top 10 features driving the `is_late` prediction, alongside the regression feature importances, for a direct side-by-side comparison of what drives delivery time vs. what drives lateness risk.

3. Customer Segmentation — Clustering

This pillar groups customers into meaningful segments using unsupervised learning (no target label — the algorithm finds structure in the data on its own).

Design Decisions

- Segmentation is done on `customer_unique_id`, not `customer_id` — in this dataset, `customer_id` is generated per order, so the same real shopper re-ordering would otherwise show up as a different "customer" each time.
- Features use a classic RFM (Recency, Frequency, Monetary) approach, extended with two Olist-specific signals: `avg_review_score` (satisfaction proxy) and `avg_delivery_days` (logistics-experience proxy).

A real quirk of this dataset: about 97% of customers have placed only a single order. This is documented and expected, not a bug — it means frequency doesn't do much heavy lifting in the segments, which mostly separate customers by spending, satisfaction, and delivery experience instead.

3.1 KMeans Clustering

`k = 2` through `7` was tested, tracking both Inertia (elbow method) and Silhouette Score (cluster separation quality, computed on a 5,000-point sample for speed):

k	Inertia	Silhouette Score
2	376,874	0.5946
3	291,861	0.3633
4	234,699	0.2661
5	201,157	0.2572

k	Inertia	Silhouette Score
6	176,864	0.2662
7	163,197	0.2374

The silhouette score is technically highest at $k=2$ — but that split just separates "ordered more than once" from "ordered exactly once" (97% vs. 3% of customers), which isn't an actionable business segmentation on its own. $k=4$ sits right at the elbow of the inertia curve and produces four genuinely distinct, useful segments, so $k=4$ was chosen as final.

Final KMeans Segments ($k=4$)

Cluster	Size	Recency (days)	Frequency	Monetary (R\$)	Avg Review	Avg Delivery (days)
0	2,801	219.29	2.11	308.59	4.21	11.88
1	14,353	231.51	1.00	195.76	1.66	21.91
2	42,952	121.85	1.00	151.25	4.65	9.48
3	33,244	389.50	1.00	157.94	4.60	11.28

Final KMeans ($k=4$) silhouette score: 0.2661. Reading the profile: Cluster 2 is the largest and healthiest group — fast delivery, high satisfaction, recent activity. Cluster 1 stands out as the clear at-risk group — the worst average review score (1.66) and by far the slowest delivery (21.91 days), a strong signal that slow logistics is driving dissatisfaction. Cluster 3 is a large, satisfied but dormant group (highest recency, i.e. longest time since last purchase). Cluster 0, though smallest, has the highest average frequency and monetary value — the closest thing to a "repeat customer" segment in a dataset where repeat purchases are rare.

3.2 DBSCAN Clustering

DBSCAN groups densely-packed points together and labels sparse, isolated points as noise (label = -1) rather than forcing every point into a cluster, unlike KMeans. The neighborhood radius (eps) was chosen using a k-distance plot (distance to each point's 5th nearest neighbor, sorted) — the elbow sat around $\text{eps} \approx 0.7$, used with $\text{min_samples}=5$.

Metric	Value
Clusters found (excluding noise)	8
Noise points	452 (0.5% of all customers)
Silhouette score (excl. noise)	0.5601

DBSCAN's cluster sizes were highly uneven: one giant cluster of 90,252 "normal" customers, a second cluster of 2,488, and six tiny micro-clusters of 5-128 points each — plus 452 points flagged as noise.

3.3 KMeans vs. DBSCAN — Which One to Use

DBSCAN's silhouette score (0.5601) is higher than KMeans's (0.2661) at first glance, but that comparison is a bit misleading: DBSCAN's number only counts its one giant "normal" cluster plus a few genuinely tight micro-clusters, after quietly discarding the awkward, spread-out points as noise. KMeans doesn't get that luxury — it must assign every customer somewhere, including messy in-between cases, which naturally costs it some silhouette score.

Verdict: for an actionable, everyone-gets-a-segment business tool, KMeans's 4 segments are more useful. DBSCAN, on the other hand, is genuinely great at a different job — automatically flagging the unusual customers (its noise points) for manual review.

Both sets of clusters were visualized in 2D using PCA (Principal Component Analysis), which compressed the 5 original features down to the 2 directions capturing the most variance — together explaining 50.7% of total variance, enough for a faithful, if simplified, 2D picture of the cluster structure.

4. NLP — Review Text Analysis

This pillar works directly with the free-text review `_comment_message` field (written in Portuguese) to classify sentiment and explore what customers are actually talking about.

4.1 Sentiment Classification

Reviews were labeled positive/negative from their star rating, cleaned, and converted into TF-IDF features (max 3,000 vocabulary terms). Since positive reviews outnumber negative ones (~71% vs. ~29%), a Logistic Regression classifier was trained with `class_weight="balanced"`, the same imbalance-handling approach used for the `is_late` model.

Metric	Score
Accuracy	0.9037
Precision	0.9656
Recall	0.8956
F1 Score	0.9293

This is a strong result: TF-IDF + Logistic Regression is a lightweight, fast, and highly interpretable baseline, and here it performs very well — likely because sentiment in these reviews is expressed with fairly direct, repeated vocabulary (praise words vs. complaint words) rather than subtle or sarcastic language.

4.2 Top Keywords by Sentiment

Average TF-IDF weight per word was computed separately for positive and negative training reviews, surfacing the vocabulary that most distinguishes each class.

Positive reviews lean on: produto, bom (good), prazo (deadline/on time), antes (before/early), recomendo (recommend), entrega (delivery), chegou (arrived), ótimo (great), excelente (excellent), qualidade (quality), gostei (I liked it), rápida (fast) — praise plus fast delivery.

Negative reviews cluster around: ainda (still), aguardando (waiting), recebi (received), comprei (I bought), veio (came), nao (no/not), pedido (order), loja (store), dia (day) — overwhelmingly about delivery problems and non-arrival, not product quality.

This matches the finding from the Supervised Learning module too: delivery experience is clearly a major driver of customer sentiment on this platform, not the products themselves.

4.3 Word2Vec (Exploratory) & Named Entity Recognition

A Word2Vec model was trained separately as an optional, exploratory step (per the project brief) to show what a denser word representation captures — unlike TF-IDF, which treats every word as an independent,

disconnected column, Word2Vec learns vectors so that words used in similar contexts end up with similar vectors (e.g. "bom" and "ótimo" being close together).

A pretrained spaCy Portuguese NER pipeline was also run on a sample of raw reviews to detect people, organizations, and locations mentioned in the text. The honest result: entities were genuinely rare (found in only 4 of 20 sampled reviews), and some detections were false positives — expected, since Olist reviews are short, informal, and often ALL-CAPS or unpunctuated, unlike the well-structured news-style text most pretrained NER models are trained on.

5. Time-Series Sales Forecasting

Orders were aggregated into a monthly total-sales series, checked for stationarity, and differenced as needed before three forecasting approaches were trained and compared on held-out test months.

- ARIMA — auto_arima searches (p, d, q) combinations and picks the one with lowest AIC; no seasonal component
- SARIMA — same idea, extended with a seasonal component (period m=12) to capture yearly repeating patterns
- Prophet — designed to be more "automatic" about trend and seasonality, and handles irregular series gracefully

Time-Series Model Comparison Table

Model	MAE (R\$)	RMSE (R\$)
ARIMA	275,897	281,940
SARIMA	275,897	281,940
Prophet	349,634	354,166

SARIMA's chosen seasonal order came back as (0,0,0) — with only 17 training months (not even a year and a half), auto_arima correctly concluded there wasn't enough history to reliably detect a 12-month seasonal pattern, so SARIMA collapsed to being numerically identical to plain ARIMA. This is the model honestly refusing to invent a pattern it can't support with the available data, not a bug.

Why ARIMA (and SARIMA) Won

Both ARIMA and Prophet overshoot the actual test months, because sales growth genuinely slowed down in 2018 after a steep 2017 ramp-up, and both models extrapolate a trend. ARIMA's chosen model included a negative autoregressive coefficient, which pulled its forecast back toward recent values rather than projecting the steep trend forward in a straight line. Prophet's trend component, by contrast, continued the growth trajectory more literally, leading to a bigger overshoot.

ARIMA was selected as the final forecasting model.

Final 12-Month Forecast

The chosen ARIMA model was refit on all 20 months of available data and used to forecast 12 months ahead — the top end of the requested 6-12 month range. Sales are projected to keep gradually rising, continuing the plateau-with-slight-growth pattern seen through mid-2018, rather than either collapsing or resuming steep growth.

Honesty check on the forecast: near-term months (1-3 out) are the most reliable, matching the short-horizon test-set evaluation above. The 95% confidence band widens substantially further out — by month 12 it spans

roughly R\$1.04M to R\$2.17M — reflecting genuine uncertainty from having only 20 months of history to learn from, and the fact that yearly seasonality (e.g. a possible November spike) could not be reliably confirmed or ruled out with the data available.

6. End-to-End ML Pipelines with Cross-Validation

The top-performing regression and classification models from Sections 1-2 were rebuilt as complete scikit-learn Pipeline objects (bundling preprocessing + model as one unit) and re-validated using 3-fold cross-validation on the training set — giving a mean and standard deviation per metric across different data slices, a more robust check than a single train/test split.

Tuning note: the Random Forest models here use `max_features="sqrt"` and `max_depth=10` — standard regularization choices that speed up training and reduce overfitting risk, applied consistently to both the Regressor and Classifier.

6.1 Regression — Cross-Validated Comparison

Model	CV MAE	CV RMSE	CV R ²
Random Forest Regressor	5.156	8.313	0.252
Linear Regression	5.279	8.472	0.223

6.2 Classification — Cross-Validated Comparison

Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV ROC-AUC
Random Forest	0.790	0.201	0.533	0.292	0.744
Logistic Regression	0.640	0.135	0.632	0.223	0.695

6.3 Final Model Selection (Deliverable)

Regression: Random Forest Regressor wins on every cross-validated metric (CV R² = 0.252 vs. Linear Regression's 0.223, with similarly lower MAE and RMSE), and this holds up on the final held-out test set (R² = 0.268, MAE = 5.128, RMSE = 7.969). This is consistent with Section 1 — non-linear, tree-based models handle this problem better than a straight line can.

Classification — a genuinely interesting side-finding: Random Forest again wins on ROC-AUC (0.744 vs. Logistic Regression's 0.695) and accuracy (0.790 vs. 0.640). But notice something different from Section 2 here — this Random Forest's recall (0.533) is much healthier than the near-zero recall (0.202 in the CV table, and as low as 0.06 in an earlier unrestricted run) seen previously.

The difference is `max_depth=10`: capping tree depth stops the model from carving out tiny, majority-class-only leaves that ignore the rare "late" cases entirely. Regularizing the model didn't just make it faster — it made it substantially fairer to the minority class it was previously ignoring. Logistic Regression still edges it out on recall (0.632), so the same business-priority framing from Section 2 still applies: use Random Forest for the best overall ranking/accuracy, Logistic Regression when catching more true late orders matters most — but the gap between them is now much smaller than it was with an unregularized model.

Final Held-Out Test Set Performance

Pipeline	Metric	Score
Regression (Random Forest)	MAE	5.128
Regression (Random Forest)	RMSE	7.969
Regression (Random Forest)	R ²	0.268
Classification (Random Forest)	Accuracy	0.780
Classification (Random Forest)	Precision	0.188
Classification (Random Forest)	Recall	0.535
Classification (Random Forest)	F1	0.278
Classification (Random Forest)	ROC-AUC	0.745

Both final pipelines are single, self-contained objects — each one bundles its own preprocessing recipe with its trained model. That means either one could be saved (e.g. with `joblib.dump`) and reused later on brand-new orders without rewriting a single line of preprocessing code.

7. Summary of Winning Models

Pillar	Winning Model	Key Metric
Regression (delivery time)	Random Forest Regressor	R ² = 0.268 (test set)
Classification (late delivery)	Random Forest (context-dependent)	ROC-AUC = 0.745 / Recall = 0.535
Clustering (customer segments)	KMeans (k=4)	Silhouette = 0.2661
NLP (review sentiment)	Logistic Regression + TF-IDF	F1 = 0.929
Time-Series (sales forecast)	ARIMA	RMSE = R\$281,940

Across all four pillars, a consistent theme emerges: simple linear baselines are competitive and sometimes preferable for interpretability or recall-sensitive tasks, but tree-based ensembles (Random Forest) generally win on raw predictive accuracy whenever the underlying relationship is non-linear. Where uncertainty is inherent — delivery logistics, sparse text entities, or a short 20-month sales history — every section was deliberately honest about the limits of what the data could support, rather than overstating model confidence.

Full notebook and code: <https://github.com/rahulkantode26/Olist-E-Commerce-Data-Analysis-Project>